SYSTEM DESIGN

# Bloom Filters Visual Guide

Ask 'is it in the set?' in constant memory: bits, hashes, and honest false positives.

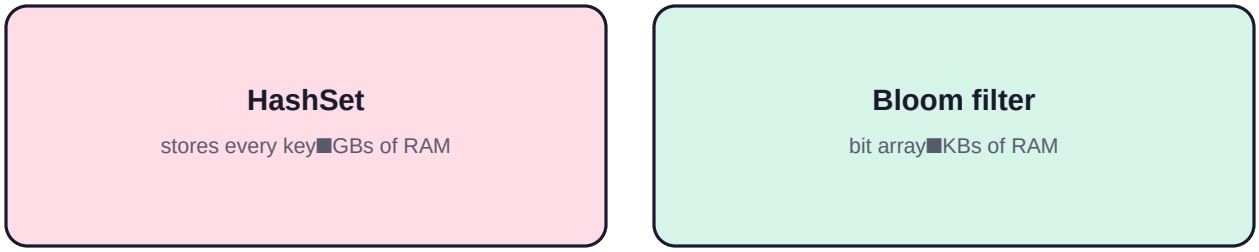
**ANIMATED WALKTHROUGH**

Step through pages to play the animation

STEP 1/6

# The Membership Question

Billions of keys, megabytes of RAM: 'have we seen this?' without storing it.




- Use when: the SET is huge, memory is scarce, and 'maybe' is an acceptable first answer.
- The deal: 'definitely not' is certain; 'probably yes' needs a (rare) double-check against the real store.

STEP 2/6

# Bit Array + k Hashes

Add: hash the item k times, set those bits. Query: check those bits.

add('apple'): h1=2, h2=7, h3=13 -> set bits 2, 7, 13

query('apple'): bits 2,7,13 all set -> PRESENT (maybe)

query('mango'): bit 5 unset -> ABSENT (certain)

STEP 3/6

# False Positives, Never False Negatives

Bits set by OTHER items can fake a 'present' -- but an unset bit never lies.

**False positive**

'mango' hashes to bits■set by apple+pear■-> wrong 'maybe'

**Never false negative**

any unset bit■-> definitely absent■-> mathematically certain

- Adding never UNSETS bits, so a true member's bits are always all set: negatives are impossible to miss.
- False positive rate is tunable: more bits per item and optimal k drive it down exponentially.
- Design pattern: bloom says 'maybe' -> check the database; bloom says 'no' -> skip the I/O entirely.

STEP 4/6

# Sizing Math

Two knobs: bits per item (m/n) and hash count (k).

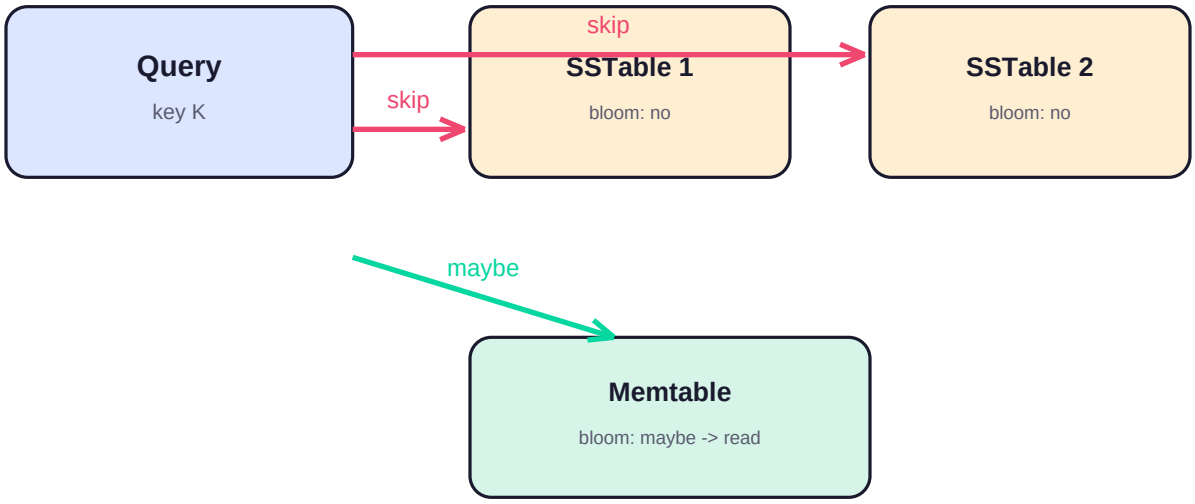
```
p ~= (1 - e^(-k*n/m))^k        k_opt = (m/n) * ln 2
```

- m/n = bits per element. 10 bits/item with optimal k ~= 7 -> p ~= 1% false positives.
- Want 0.1%? Use ~15 bits/item. 1% is plenty for 'skip a disk read'.
- Rule of thumb: 10 bits per item, k=7 -- memorize it, it covers most designs.
- k too HIGH: you fill the array faster (more FPs). k too LOW: you do not use the bits well.

STEP 5/6

# Cassandra and BigTable Use

One bloom filter per SSTable: skip disk reads for keys that are not there.




A read touches MANY SSTables; without blooms every read fans out to disk. Blooms make misses nearly free.

STEP 6/6

# Beyond the Classic

When you need deletes, counting, or fewer hashes.

### Counting Bloom

counters instead of bits -- decrement on delete (4x memory)

### Cuckoo filter

supports delete, smaller for low FP rates, needs bucketed table

### Quotient filter

cache-friendly, mergeable -- for LSM-tree metadata

### XOR filter

static sets only, but ~1.23x smaller and faster than Bloom

STEP ✓I✓

# Key Takeaways

## Bloom Filters — Remember

- A bit array + k hashes answers set membership in tiny constant memory.
- False positives are possible and tunable; false negatives are impossible by construction.
- Size it: ~10 bits/item and k=7 gives ~1% false positives -- the standard recipe.
- LSM stores keep one filter per SSTable so reads skip disk for absent keys.
- Need deletes or smaller footprints? Counting Bloom, Cuckoo, and XOR filters extend the idea.

**Keep exploring**

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev